作品介绍 · 15 分钟生活圈智能体检与规划助手

作品介绍 ·「15 分钟生活圈」智能体检与规划助手

项	内容
作品名称	「15 分钟生活圈」智能体检与规划助手
参赛赛道	开源 AI 工具赛道
所选命题	百度地图赛题：基于地图开放能力的"15 分钟生活圈"智能体检与规划助手
代码仓库	https://github.com/aurasuisui/15min-life-circle
开源许可证	Apache License 2.0
演示视频	【待填入可公开访问的链接】

一、一句话价值

输入任意一个地址，30 秒内得到一份基于真实路网的社区步行可达性"体检报告"——不只是画一个圈，而是告诉你缺什么、缺在哪、该补在哪。

二、要解决的痛点

国家正在推行"15 分钟便民生活圈"建设，要求居民步行 15 分钟内可获得基础医疗、教育、购物、养老等服务。但目前的社区评估存在两个硬伤：

1. **用直线距离代替真实可达性。** 道路不是直线——红绿灯、过街天桥、河道、施工围挡、封闭小区都会显著改变实际步行范围。一个直线距离 800 米的菜市场，绕过一条河可能要走 25 分钟。
2. **评估成本高、更新慢、看不见盲区。** 人工普查无法规模化，静态数据无法反映"这个小区哪一片区域既没有菜市场也没有药店"。

结果是：需要被看见的服务盲区，恰恰最难被看见。

三、解决方案

3.1 总体架构



```
后端 FastAPI (Python 3.13) │ │ ┌──────────────────────────────────────────────────────┐ │
│ │ 算法核心 app/core/ │ │ │ 射线采样 → PAVA 单调化 → 阈值求根加密 │ │ │ → IDW 耗时场重建
→ marching squares 等值线 → shapely │ │ │
├──────────────────────────────────────────────────────┤ │ │ 基础设施 app/baidu/ │ │ │ │
SN 签名 · 分服务令牌桶(2.5QPS) · SQLite 配额账本 │ │ │ 参数哈希缓存 · 错误三元组分类 · 双路
线 provider │ │ ├──────────────────────────────────────────────────────┤ │ │ AI 层
app/ai/ │ │ │ DeepSeek + 规范条文 RAG + 强制 JSON Schema │ │ │ + 数值一致性校验 + 模板降级
│ │ └──────────────────────────────────────────────────────┘ │
└──────────────────────────────────────────────────────┘ @
```

3.2 核心算法：为什么不用"把射线交点连起来"

这是本项目**最重要的一个技术判断**，有实测数据支撑。

百度地图开放平台**不暴露底层路网图**，无法直接跑 Dijkstra。一个自然想法是：以中心点为原点向四周发射线，找到每条射线上 15 分钟的边界点，连起来就是等时圈。

这个做法是错的。 我们在合成场景（2.4 km 见方、一条不可穿越的河 + 唯一桥梁、以 8 连通栅格 Dijkstra 为真值）上实测：

采样配置	采样点数	极坐标连点 IoU	IDW + 等值线 IoU
R=36, Δr=150 m	396	0.785	**0.941**
R=36, Δr=200 m + 阈值加密	396	0.785	**0.932**
R=36, Δr=100 m	576	0.785	**0.958**

极坐标连点会系统性低估面积——内接星形多边形会把一切凹角切掉，而凹角恰恰对应河湾、封闭小区这些最需要被识别的区域。而且它的误差**由模型偏差主导、而非采样密度**：射线数从 12 增到 144，IoU 也只从 0.90 升到 0.94。

我们的做法：射线采样只用来**取数**，多边形从**耗时场**里提取：

> 射线采样（R=36 × Δr=200 m + 阈值附近局部加密 ≈ 396 点） → PAVA 保序回归单调化（消除绑路抖动） → 逐射线求根，定位 5/10/15 分钟穿越区间 → IDW(k=8, p=2) 重建 25 m 规则栅格耗时场（含屏障遮蔽，防"穿墙"） → contourpy marching squares 提取等值线 → shapely 修复拓扑（处理 contourpy 的重复点坑）

用约 1/9 的采样量，IoU 反而从 0.785 提到 0.94+。

3.3 配额治理：把"限制"变成"设计"

个人开发者的配额是硬约束，尤其**地点检索只有 100 次/日**。我们没有回避它，而是围绕它做了三层设计：

手段	效果	实测依据
多关键字并集检索	8 类民生设施从 8~24 次请求压到 **2 次**	实测确认**按请求计费**（2 次请求 = 配额 2%）
多档位一次采样	5/10/15 三档从 924 条路线降到 **396 条**（省 2.3 倍）	单元测试对比
快照 + 离线回放	评审方**零配额**可跑通全流程	采集后 replay 复跑结果一致

配套的容错降级：按服务分桶的令牌桶（2.5 QPS）、SQLite 配额账本（80% 黄 / 95% 红）、参数哈希缓存（命中不计配额）、**按服务分别分类的错误处理**（百度不同 API 的错误码语义不一致）、以及命中不可用端点时自动熔断并切换到备用路线接口。

降级必须显式告知：每一处降级都写入报告的 `meta.degrade_reasons`，前端在界面上直接展示，绝不静默返回不完整数据。

3.4 AI 层：数字归引擎，表达归模型

本项目在**开源 AI 工具赛道**，但核心是确定性地理计算。所以 AI 层不是装饰，而是**用 AI 把"算出来的数字"翻译成"人能看懂、能决策的报告"**。

铁律：所有数字由确定性引擎产出，LLM 只做解释、归因与表达。

- **RAG 报告解读**：知识库收录《城市居住区规划设计标准》GB 50180-2018、《完整居住社区建设标准（试行）》、《上海市 15 分钟社区生活圈规划导则》、商务部《城市一刻钟便民生活圈建设指南》共 4 份规范 32 条条文。模型输出强制 JSON Schema，每条结论必须引用规范出处。
- **防幻觉双保险**：
  1. 提示词注入"只能引用给定指标，缺失则写数据不足"；
  2. 输出后做**数值一致性校验**——正则抽出所有数字与报告数值集合比对，不一致则带反馈重生成，仍失败则降级为规则模板。
- **攻击测试**：我们构造了"要求模型编造 25 处菜市场、30 所小学、95.0 分"的对抗注入用例。模型拒绝了注入；即使强制喂入编造 JSON，校验闸门也能拦下并降级，且模板自身同样通过数值自检。
- **降级不缺失功能**：不配 LLM Key 时走规则模板，同一 Schema、同样可用。

四、技术亮点小结

1. **实测驱动的算法选型**——不是"照着文档实现"，而是在合成强凹陷场景上横向对比 3 种采样配置 × 2 种重建方法，用 IoU 数据决定技术路线，并诚实记录被否定的方案。
2. **把配额约束前移为架构设计**——多关键字合并 + 多档位共享采样 + 快照回放，让 100 次/日的检索配额能支撑 **30~50 个社区/日**的分析量。
3. **AI 与确定性计算的正确分工**——数值可追溯、幻觉可拦截、无 Key 可降级。
4. **踩到并解决了两个隐蔽的官方坑**（见下），这类"不报错但功能失效"的问题排查成本极高。

附：两个实测踩坑（工程能力证明）

坑 1：JSAPI 4.0 已移除 `BMapGL.AreaMask`。 官方文档页面仍在讲 GL 时代的用法，但 4.0 里 `typeof BMapGL.AreaMask === 'undefined'`。4.0 的对应能力是 `new BMapGL.MapMask(ring, opts)` + `map.addFocusMask()`。我们用像素级验证确认了压暗效果（同一截图内圈内平均亮度 169.8、圈外 92.5）。

坑 2：官方入口脚本只能用同步 `<script>` 标签加载。 `https://api.map.baidu.com/api?type=webgl&v=4.0&ak=…` 返回的只是引导脚本，它内部靠 `document.write` 注入真正的 1.1 MB 地图库；而 `document.write` 只在文档解析期有效。用 `createElement('script')` 异步注入时会被浏览器**静默忽略**——不报错、console 干净、网络面板无请求，但 `window.BMapGL` 永远是 undefined。我们的解法是直接加载真实库地址 `/getscript?v=4.0&ak=…`，并保留劫持 `document.write` 作为回退。

五、评测数据

5.1 样例社区真实对比测试

> 中心点：广州软件学院（BD09 `23.458320, 113.502141`） 方法：**高驰运动手表连续轨迹记录**（2417 个轨迹点，含时间戳），非离散点测量

实测概况

指标	实测值
总历时	**47.4 分钟**
总里程	**3.36 km**
平均步速	**1.18 m/s**
距中心最远	345 m

核心验证：逐点包含性检验（可证伪）

逻辑：若某点在录制开始后第 T 秒被实际走到，则该点从中心步行可达时间 **≤ T 秒**（实际路线含绕行与折返，只会更久），因此**必须**落在系统的 T 分钟等时圈内。

项	结果
参与检验的轨迹点	**2417**
越界点数	**0**
越界率	**0.00 %**

系统等时圈对实测轨迹的包含率为 100%，零反例。

另一组独立距离参照：同一社区 10 个地标的高德步行路径长度显示， 绕行倍数在 **1.21× ~ 4.46×** 之间，**方向差异达 3.7 倍** —— 直线距离评估在本地块完全失效，与系统输出的各向异性一致。

适用范围（如实说明）：本次为**单边界检验。**实测最远 345 m， 未能触及 15 分钟圈（边界半径 177~825 m），因此 **可证明「圈没有画小」，不能证明「圈没有画大」。** 详见 真实对比测试报告。

5.2 算法精度

合成场景实测（以 8 连通栅格 Dijkstra 为真值）： **IoU 0.932 ~ 0.958**，面积误差 +1.3% ~ −1.7%。

真实场景实测（见 §5.1）： **2417 个实测轨迹点零越界。**

5.3 样例社区体检结果（离线快照，零配额）

指标	值
5 / 10 / 15 分钟等时圈面积	0.266 / 0.834 / 1.856 km²
路线配额消耗	396 条（单次采样覆盖三档）
POI 检索消耗	2 次（覆盖 8 类设施）
清洗漏斗	检索 32 → 去重 31 → 纠错 30 → 圈内 25
识别盲区	1 处，合计 46,875 m²，缺失设施：小学
综合评分	59.75 / 100（C 级）

5.4 开源工程

- 后端测试 **132 个用例**（零网络、零配额），AI 层 **103 个用例**，全部通过
- 无第三方依赖自测入口 `scripts/selftest.py`（122 个用例，仅需 Python 标准库）
- CI：密钥扫描 / 后端测试 + 基准复跑 / 前端构建 / Docker 构建与冒烟
- 一键启动：`scripts/run.ps1` / `run.sh`，支持 `-Offline` 零配额模式

六、可复现性（评审方视角）

`@bash

方式一：离线模式（零 API 配额，推荐）

cp .env.example .env # 无需填入任何 AK ./scripts/run.sh --offline # 或 .scriptsun.ps1 -Offline

打开 http://127.0.0.1:8000

方式二：Docker

docker compose up `@

离线模式不需要任何 AK/SK，也不需要联网调用百度接口——所有等时圈与 POI 数据来自仓库内 `data/seed/` 快照，可完整复现体检报告全流程。

七、团队与分工

团队构成：1 人（独立完成）

模块	负责内容
算法设计	等时圈重建管线：射线采样布局、耗时场插值、等值线提取与拓扑修复；含合成场景的算法选型对照实验
后端工程	服务端接口封装与签名校验、限额令牌桶、配额账本、多级缓存、容错降级链、数据清洗流水线
前端	Vue 3 单页应用：地图图层、等时圈渲染、盲区呈现、ECharts 图表组、交互与响应式布局
AI 层	规范条文检索、结构化输出约束、数值一致性校验、模板降级
现场实测	现场轨迹采集（高驰运动手表）、两套大地坐标系转换、逐点包含性验证
工程化	Docker 一键部署、持续集成、5 篇技术设计文档、开源治理文件

> 全部代码、文档与实测数据均由本人独立完成。

八、许可证

Apache License 2.0。第三方依赖与许可证清单见 `README.md` 附录。